Pest Control Management System - Project Report

Project Overview

This report provides a comprehensive analysis of the **Pest Control Management System**, a full-stack web application built using **ASP.NET Core Web API in C#**.

The system manages pest control operations including: - Customer management - Booking scheduling - Technician assignments - Pest type tracking - Treatment records - Inspection reports

It also features an **AI-powered chat interface using Claude API**, a responsive web frontend, and a relational database backend.

System Architecture

High-Level Architecture

The application follows a **layered architecture pattern**:

- **Presentation Layer**: Single Page Application (SPA) with HTML, CSS, and JavaScript
- **Application Layer**: ASP.NET Core Web API controllers handling HTTP requests
- **Domain Layer**: Business logic and data models
- **Infrastructure Layer**: Data access repositories and external service integrations

Design Patterns Implemented

Repository Pattern

The system uses the Repository Pattern for data access abstraction:

- Interfaces define contracts (e.g., `ICustomerRepository`)
- Multiple implementations:
  - SQL-based for production
  - Static in-memory for testing
- Controllers depend on abstractions rather than implementations (Dependency Inversion Principle)

Dependency Injection (DI)

- Centralized service registration in `Program.cs`
- Constructor-based dependency injection
- Singleton lifetime for shared services (repositories, AI agent)

Singleton Pattern

- Repositories and AI agent registered as singletons
- Ensures a single instance throughout application lifecycle
- Enables efficient and thread-safe access

Database Design

Schema Overview

The system uses **Microsoft SQL Server** with the following tables:

Customers Table

```sql
CREATE TABLE Customers (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL,
    Address NVARCHAR(255) NOT NULL,
    Phone NVARCHAR(20) NOT NULL,
    Email NVARCHAR(100) NOT NULL,
    PropertyType NVARCHAR(50) NOT NULL
);
```

PestTypes Table

```sql
CREATE TABLE PestTypes (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL,
    Category NVARCHAR(50) NOT NULL,
    Description NVARCHAR(500) NOT NULL,
    RiskLevel NVARCHAR(20) NOT NULL
);
```

Technicians Table

```sql
CREATE TABLE Technicians (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL,
    Specialisation NVARCHAR(100) NOT NULL,
    Phone NVARCHAR(20) NOT NULL,
    Email NVARCHAR(100) NOT NULL,
    Available BIT NOT NULL DEFAULT 1
);
```

Bookings Table

```sql
CREATE TABLE Bookings (
    Id INT PRIMARY KEY IDENTITY(1,1),
    CustomerId INT NOT NULL,
    PestTypeId INT NOT NULL,
    TechnicianId INT NOT NULL,
```

```sql
    Date NVARCHAR(20) NOT NULL,
    Time NVARCHAR(10) NOT NULL,
    Status NVARCHAR(30) NOT NULL,
    Location NVARCHAR(255) NOT NULL,
    Notes NVARCHAR(500) NOT NULL,
    FOREIGN KEY (CustomerId) REFERENCES Customers(Id),
    FOREIGN KEY (PestTypeId) REFERENCES PestTypes(Id),
    FOREIGN KEY (TechnicianId) REFERENCES Technicians(Id)
);
```

Treatments Table

```sql
CREATE TABLE Treatments (
    Id INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    Method NVARCHAR(100) NOT NULL,
    TargetPestTypeId INT NOT NULL,
    SafetyInfo NVARCHAR(500) NOT NULL,
    FOREIGN KEY (TargetPestTypeId) REFERENCES PestTypes(Id)
);
```

InspectionReports Table

```sql
CREATE TABLE InspectionReports (
    Id INT PRIMARY KEY IDENTITY(1,1),
    BookingId INT NOT NULL,
    Findings NVARCHAR(1000) NOT NULL,
    Recommendations NVARCHAR(1000) NOT NULL,
    FollowUpNeeded BIT NOT NULL DEFAULT 0,
    ReportDate NVARCHAR(20) NOT NULL,
    FOREIGN KEY (BookingId) REFERENCES Bookings(Id)
);
```

Database Connection and Security

- Parameterized queries prevent SQL injection
- Connection string configured in `Program.cs`
- Proper connection handling using `using` statements
- ADO.NET used for direct SQL execution

Backend Implementation

Application Entry Point (Program.cs)

- Configures dependency injection container
- Registers repositories as singletons
- Integrates Claude API agent
- Sets up middleware and routing

Controllers

Standard CRUD Controllers

- CustomersController
- BookingsController
- TechniciansController
- PestTypesController
- TreatmentsController
- InspectionReportsController

Specialized Controllers

- AgentController (AI chat)
- SearchController (cross-entity search)
- DashboardController (statistics)

Data Access Layer

Repository Interfaces

- ICustomerRepository
- IBookingRepository
- ITechnicianRepository
- IPestTypeRepository
- ITreatmentRepository
- IInspectionReportRepository

SQL Repository Implementation

- Built using ADO.NET
- Parameterized queries for security
- Efficient connection handling

Static Repository Implementation

- In-memory storage for testing
- Same interface implementations as SQL repositories

AI Integration (Claude API)

PestControlAgent Architecture

Agent Arms

1. CustomerSearch
2. TechnicianAvailability
3. TreatmentRecommendation
4. BookingLookup

5. PestInfo
6. ReportSummary
7. DashboardStats

Intent Matching Algorithm

- Keyword-based scoring system
- Multi-word keywords weighted higher
- Highest scoring arm handles request
- Database context fetched before AI call

Claude API Integration

- HTTP calls via HttpClient
- Async processing
- Context-aware responses

AgentController

- POST /api/agent
- GET /api/agent/arms

Frontend Implementation

Single Page Application (SPA)

- Built with HTML, CSS, JavaScript
- Dynamic navigation without reloads

Key Components

Navigation

- loadPage() routing
- Dynamic content rendering

Data Management

- fetchJson(), postJson(), deleteReq()
- Real-time updates

AI Chat Widget

- IIFE structure
- Typing animations
- XSS protection

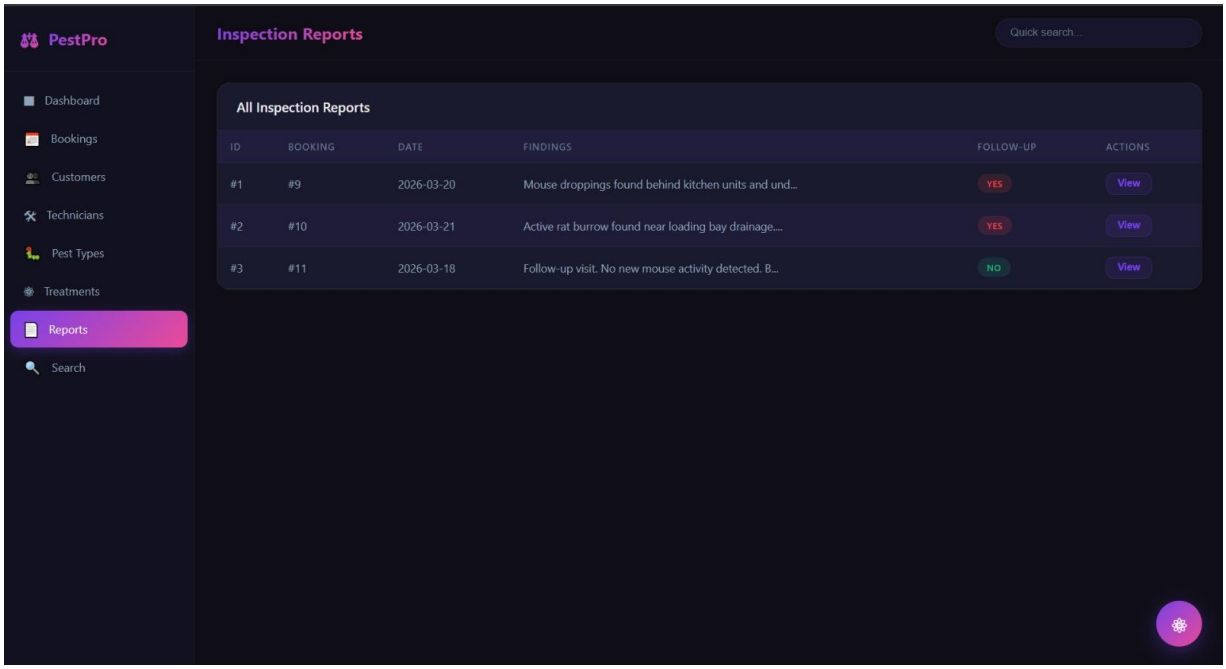
Form Handling

- Dynamic forms
- Dropdown population

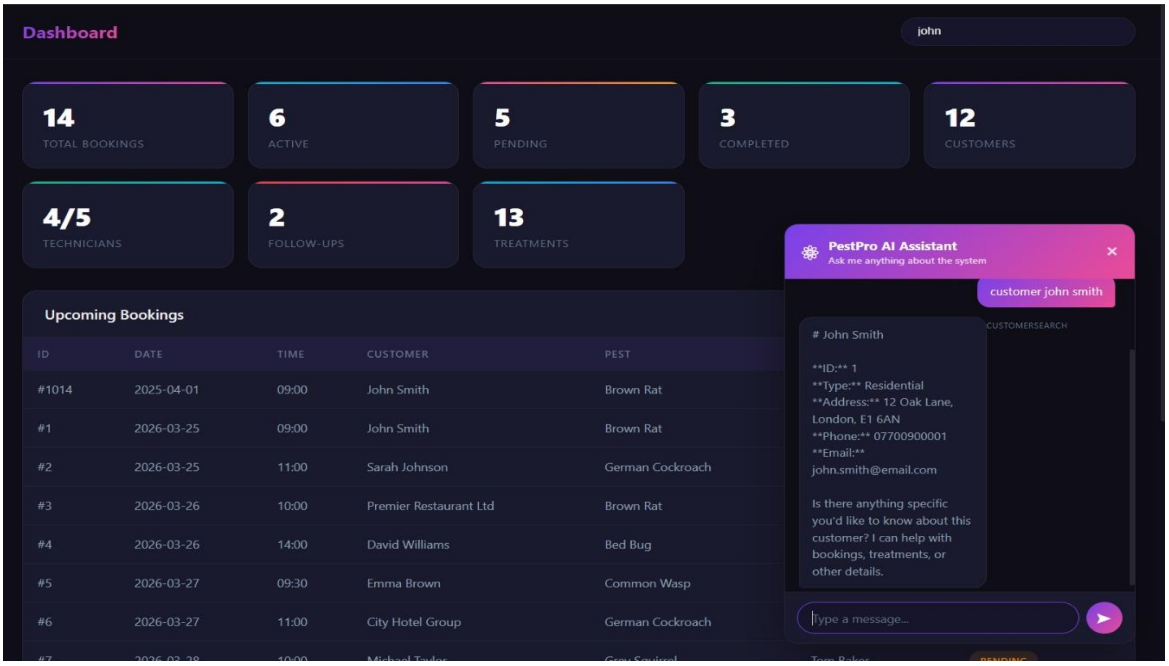
- Validation and error handling

Finished User Interface

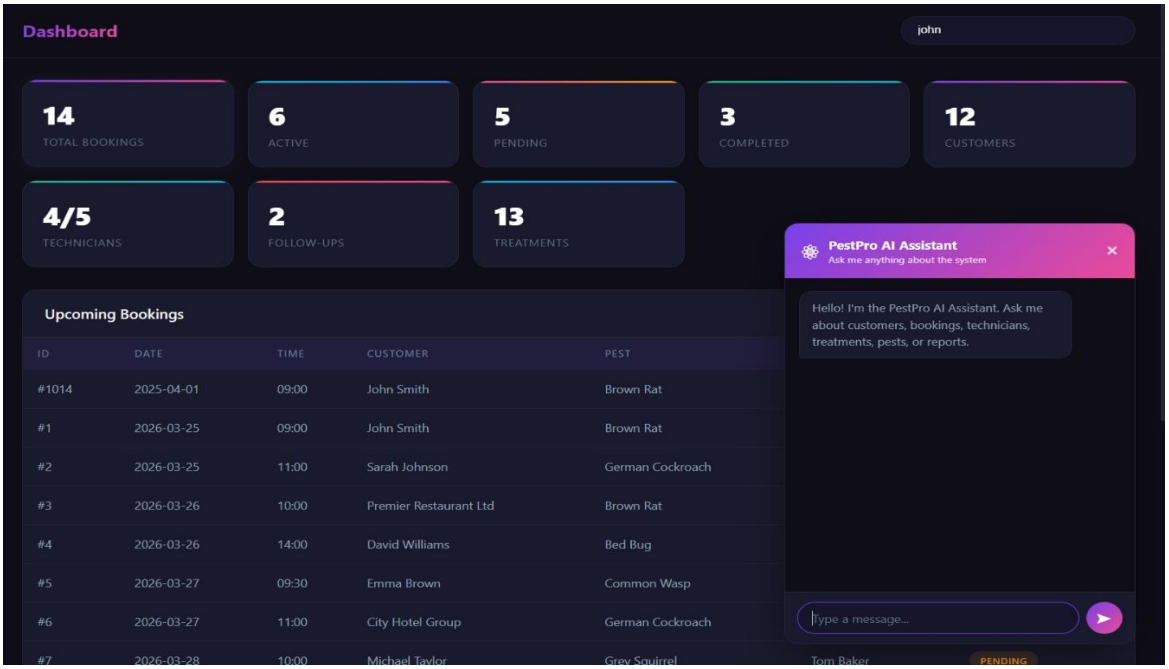
Inspection report page



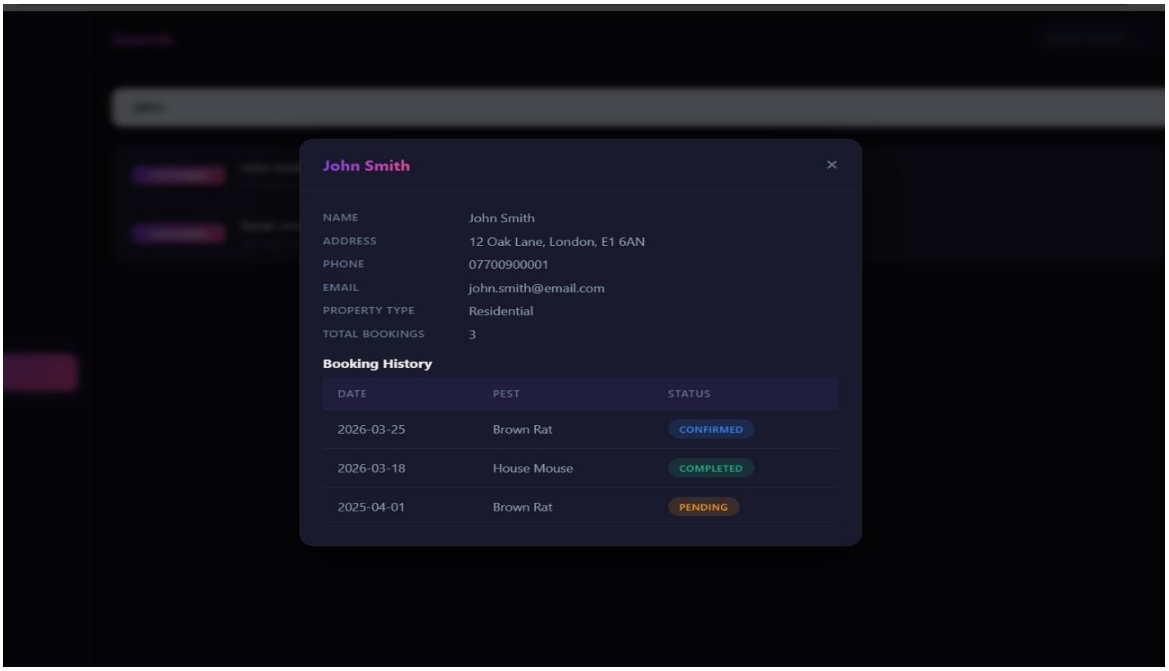
Dashboard



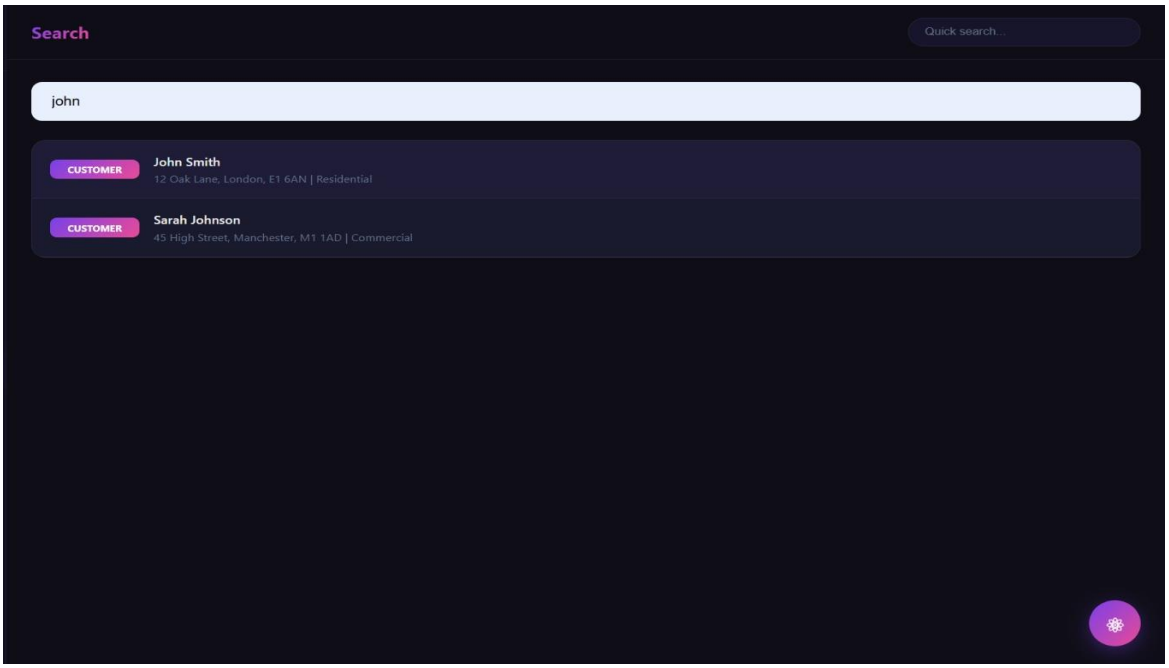
PestPro AI assistnat



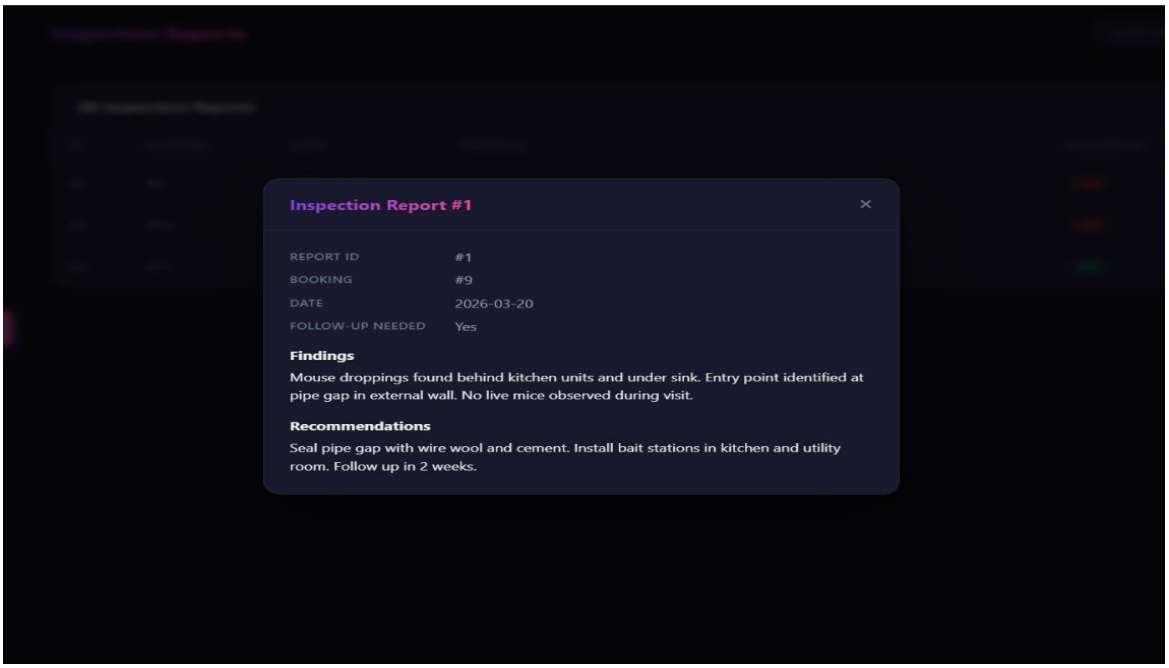
Booking information



Search



Inspection report



Inspection Report #1

REPORT ID	#1
BOOKING	#9
DATE	2026-03-20
FOLLOW-UP NEEDED	Yes

Findings

Mouse droppings found behind kitchen units and under sink. Entry point identified at pipe gap in external wall. No live mice observed during visit.

Recommendations

Seal pipe gap with wire wool and cement. Install bait stations in kitchen and utility room. Follow up in 2 weeks.

Treatment history

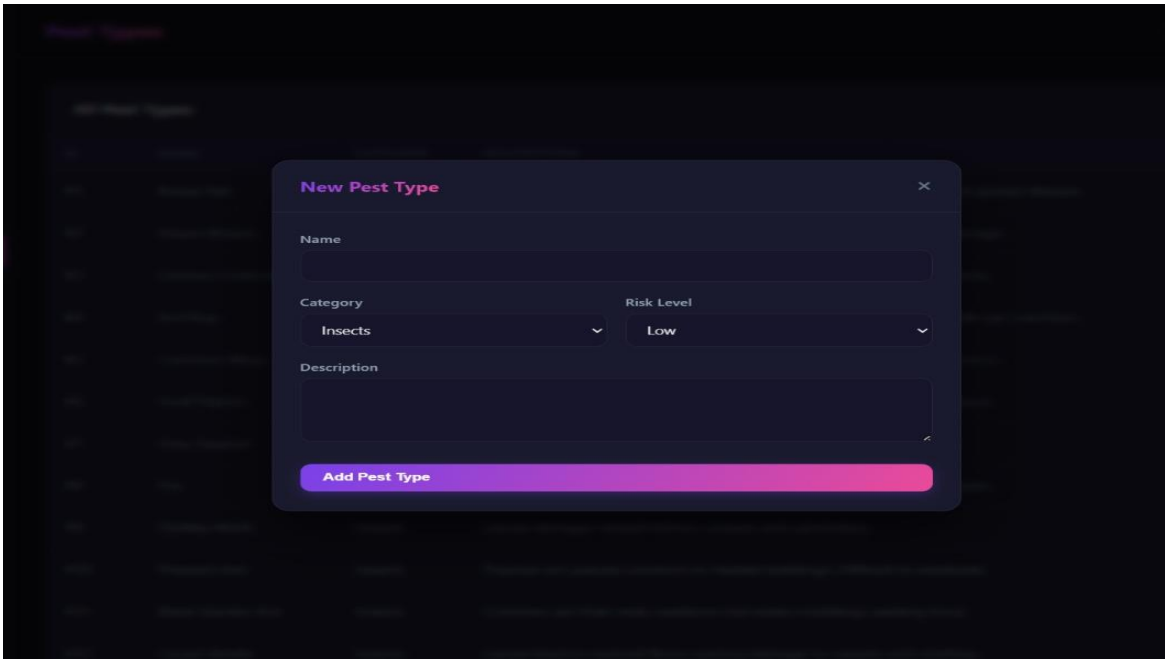
Treatments

Quick search...

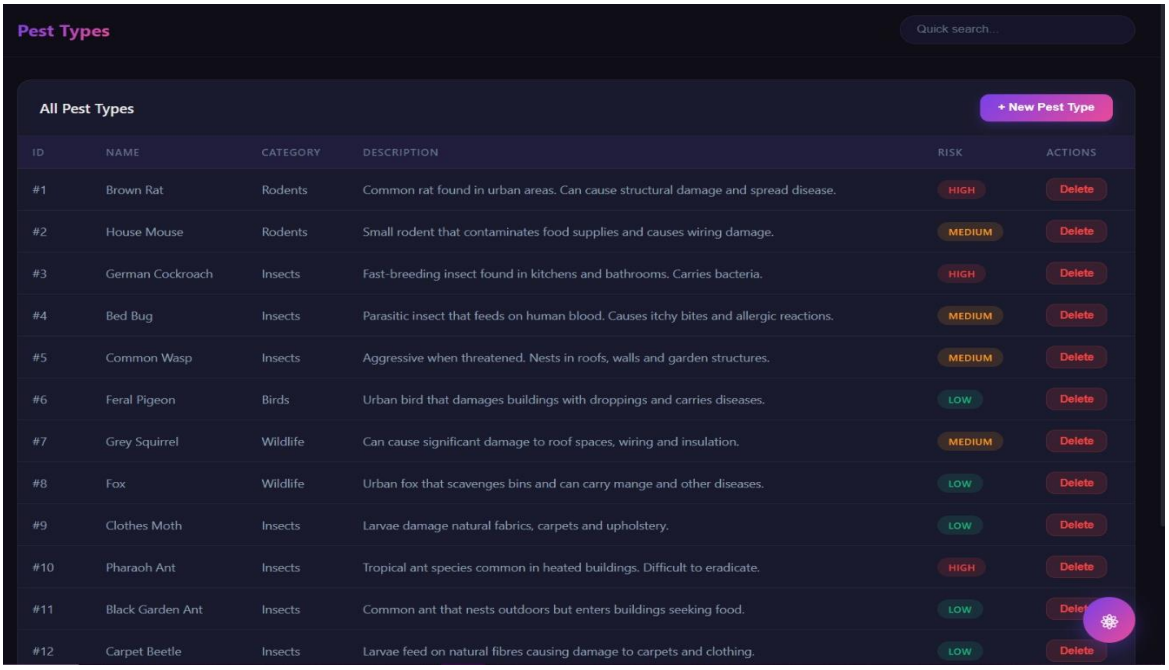
All Treatments

ID	PRODUCT	METHOD	TARGET PEST	SAFETY INFO
#1	RodentBlock Pro	Bait Station	Brown Rat	Keep away from children and pets. Use tamper-resistant bait ...
#2	MouseGuard Snap	Mechanical Trap	House Mouse	Set traps along walls away from foot traffic. Check traps da...
#3	InsectaClear Gel	Gel Bait Application	German Cockroach	Apply in small dots in cracks and crevices. Avoid food prepa...
#4	BedBugHeat Treatment	Thermal Remediation	Bed Bug	Room temperature raised to 56C for sustained period. Remove ...
#5	WaspNest Powder	Insecticidal Dust	Common Wasp	Apply at dusk when wasps are less active. Wear protective eq...
#6	PigeonNet System	Bird Netting	Feral Pigeon	Professional installation required. Stainless steel fixings....
#7	SquirrelCage Trap	Live Capture Trap	Grey Squirrel	Check traps twice daily minimum. Bait with peanut butter. Le...
#8	FoxDeter Spray	Scent Deterrent	Fox	Apply around perimeter weekly. Non-toxic formula. Reapply af...
#9	MothCedar Blocks	Natural Repellent	Clothes Moth	Place in wardrobes and drawers. Replace every 6 months. Sand...
#10	AntBait Station	Bait Station	Pharaoh Ant	Place near ant trails. Do not disturb stations. Allow 2-3 we...
#11	BugBuster spray	surface spray application	Bed Bug	ventilate room for 2 hours after use
#12	SilverGuard gel	gel bait application in cracks and crevices	Silverfish	non-toxic to humans
#1011	FireAnt Killer	spray	Fire Ants	wear gloves and mask

Create new pest type

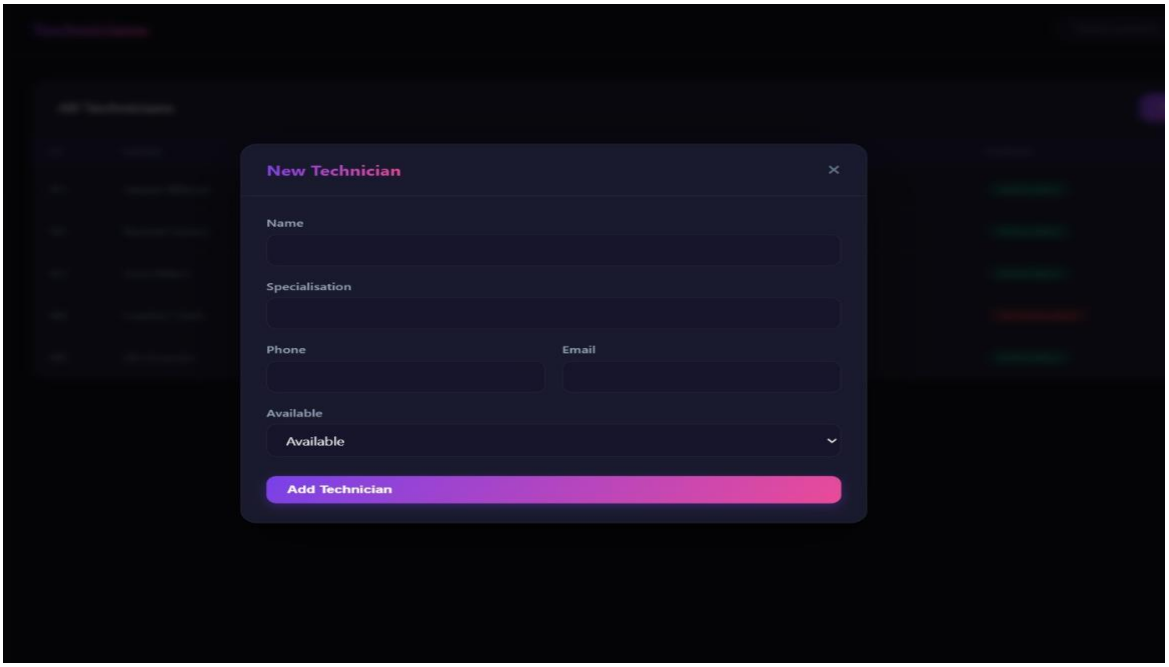


Pest types

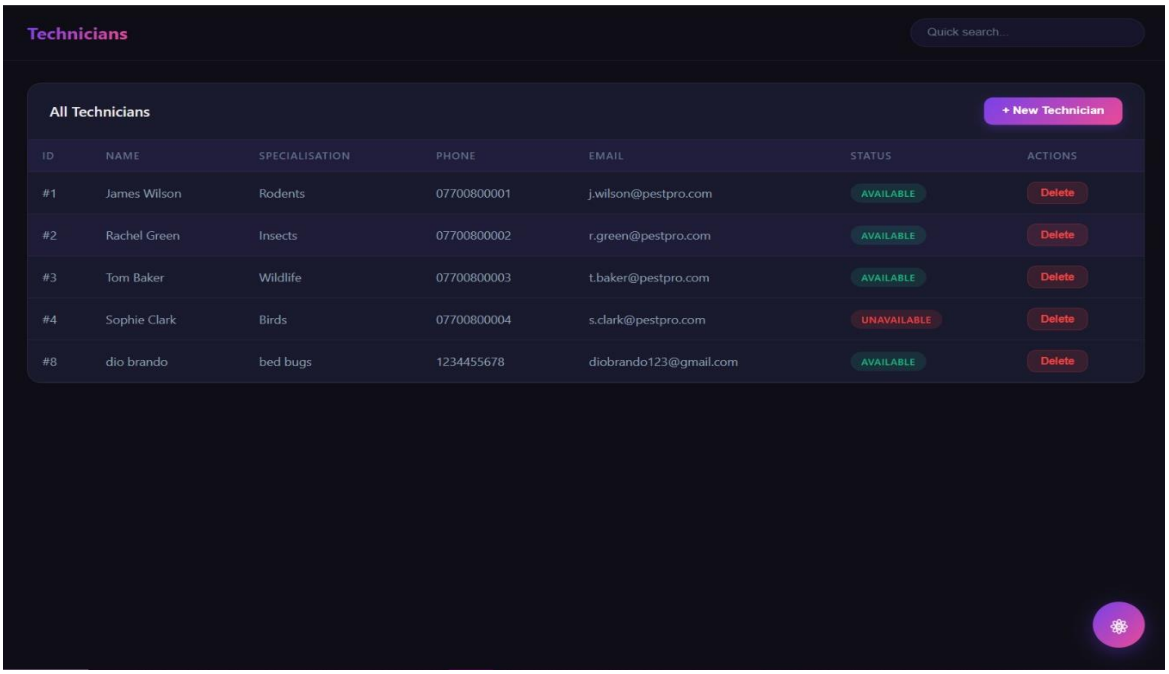


ID	NAME	CATEGORY	DESCRIPTION	RISK	ACTIONS
#1	Brown Rat	Rodents	Common rat found in urban areas. Can cause structural damage and spread disease.	HIGH	Delete
#2	House Mouse	Rodents	Small rodent that contaminates food supplies and causes wiring damage.	MEDIUM	Delete
#3	German Cockroach	Insects	Fast-breeding insect found in kitchens and bathrooms. Carries bacteria.	HIGH	Delete
#4	Bed Bug	Insects	Parasitic insect that feeds on human blood. Causes itchy bites and allergic reactions.	MEDIUM	Delete
#5	Common Wasp	Insects	Aggressive when threatened. Nests in roofs, walls and garden structures.	MEDIUM	Delete
#6	Feral Pigeon	Birds	Urban bird that damages buildings with droppings and carries diseases.	LOW	Delete
#7	Grey Squirrel	Wildlife	Can cause significant damage to roof spaces, wiring and insulation.	MEDIUM	Delete
#8	Fox	Wildlife	Urban fox that scavenges bins and can carry mange and other diseases.	LOW	Delete
#9	Clothes Moth	Insects	Larvae damage natural fabrics, carpets and upholstery.	LOW	Delete
#10	Pharaoh Ant	Insects	Tropical ant species common in heated buildings. Difficult to eradicate.	HIGH	Delete
#11	Black Garden Ant	Insects	Common ant that nests outdoors but enters buildings seeking food.	LOW	Delete
#12	Carpet Beetle	Insects	Larvae feed on natural fibres causing damage to carpets and clothing.	LOW	Delete

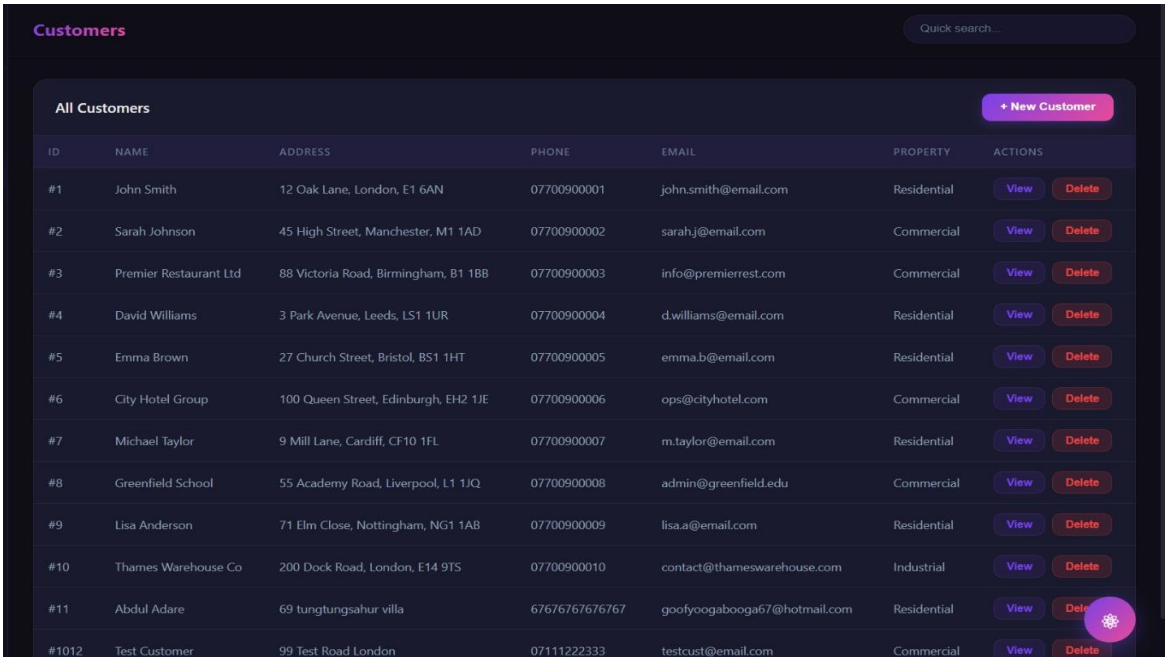
Create technician



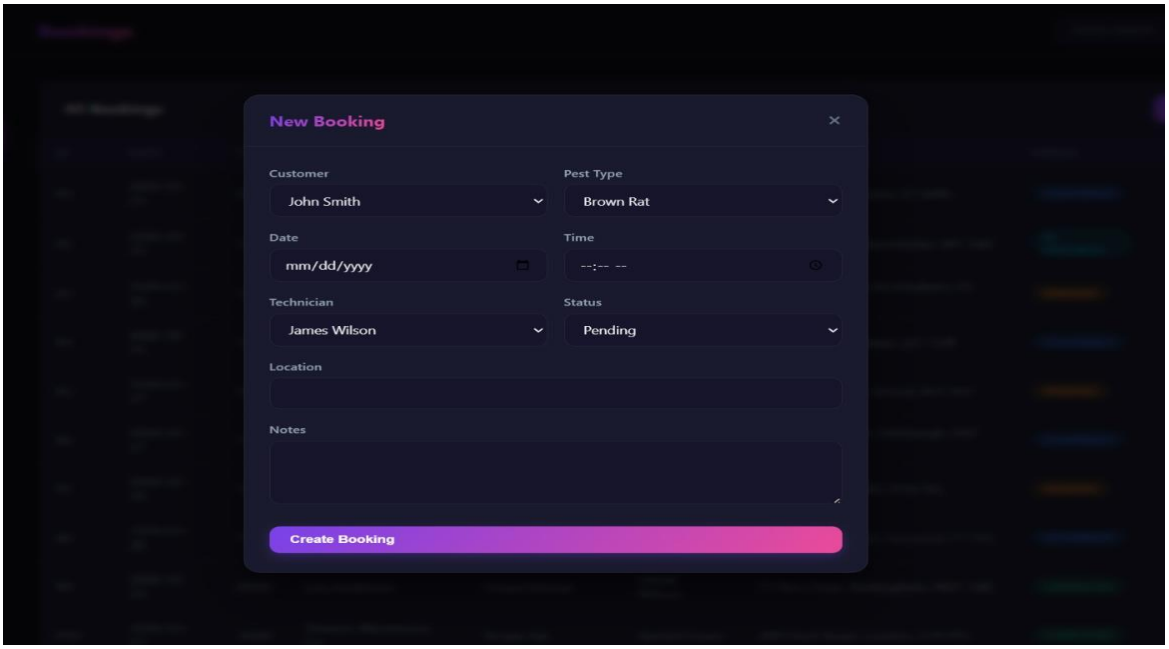
Technician list



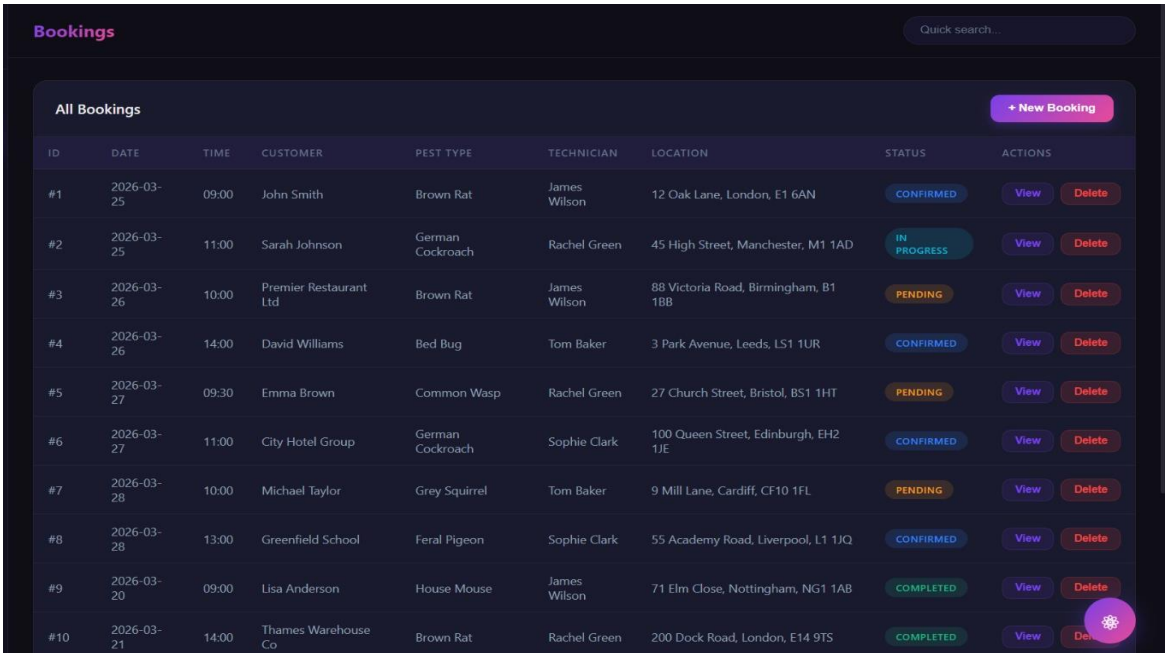
Customer list



Create new booking



Bookings list



Bookings

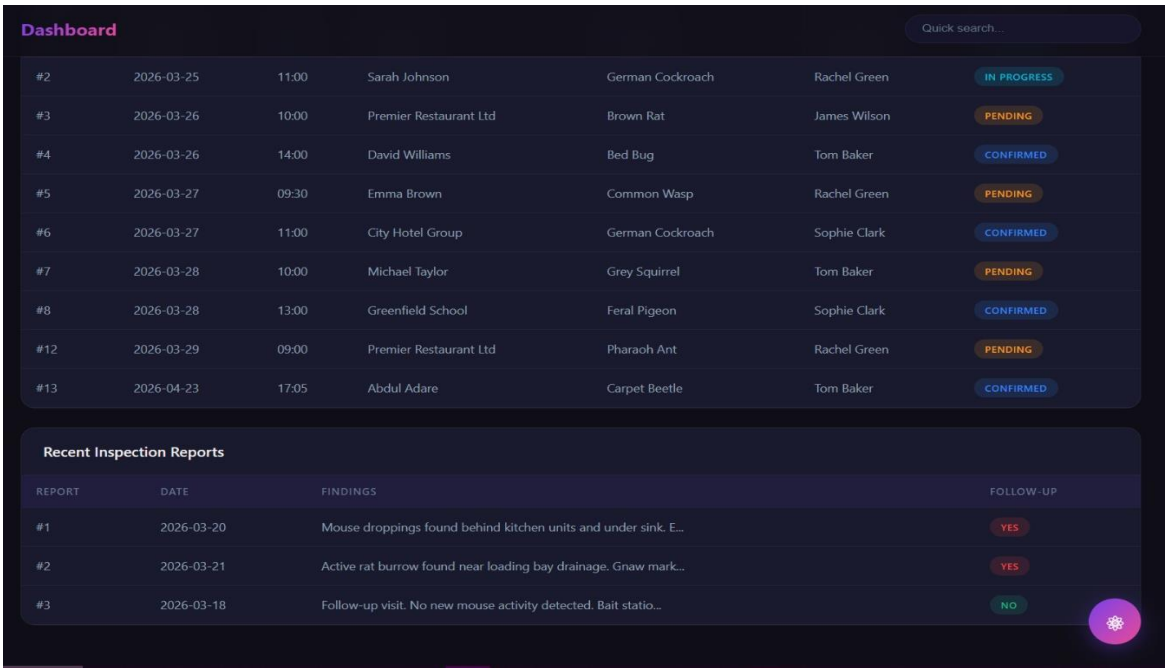
Quick search...

All Bookings

+ New Booking

ID	DATE	TIME	CUSTOMER	PEST TYPE	TECHNICIAN	LOCATION	STATUS	ACTIONS
#1	2026-03-25	09:00	John Smith	Brown Rat	James Wilson	12 Oak Lane, London, E1 6AN	CONFIRMED	View Delete
#2	2026-03-25	11:00	Sarah Johnson	German Cockroach	Rachel Green	45 High Street, Manchester, M1 1AD	IN PROGRESS	View Delete
#3	2026-03-26	10:00	Premier Restaurant Ltd	Brown Rat	James Wilson	88 Victoria Road, Birmingham, B1 1BB	PENDING	View Delete
#4	2026-03-26	14:00	David Williams	Bed Bug	Tom Baker	3 Park Avenue, Leeds, LS1 1UR	CONFIRMED	View Delete
#5	2026-03-27	09:30	Emma Brown	Common Wasp	Rachel Green	27 Church Street, Bristol, BS1 1HT	PENDING	View Delete
#6	2026-03-27	11:00	City Hotel Group	German Cockroach	Sophie Clark	100 Queen Street, Edinburgh, EH2 1JE	CONFIRMED	View Delete
#7	2026-03-28	10:00	Michael Taylor	Grey Squirrel	Tom Baker	9 Mill Lane, Cardiff, CF10 1FL	PENDING	View Delete
#8	2026-03-28	13:00	Greenfield School	Feral Pigeon	Sophie Clark	55 Academy Road, Liverpool, L1 1JQ	CONFIRMED	View Delete
#9	2026-03-20	09:00	Lisa Anderson	House Mouse	James Wilson	71 Elm Close, Nottingham, NG1 1AB	COMPLETED	View Delete
#10	2026-03-21	14:00	Thames Warehouse Co	Brown Rat	Rachel Green	200 Dock Road, London, E14 9TS	COMPLETED	View Del

Dashboard with list of inspection and any ongoing work



Dashboard

Quick search...

#2	2026-03-25	11:00	Sarah Johnson	German Cockroach	Rachel Green	IN PROGRESS
#3	2026-03-26	10:00	Premier Restaurant Ltd	Brown Rat	James Wilson	PENDING
#4	2026-03-26	14:00	David Williams	Bed Bug	Tom Baker	CONFIRMED
#5	2026-03-27	09:30	Emma Brown	Common Wasp	Rachel Green	PENDING
#6	2026-03-27	11:00	City Hotel Group	German Cockroach	Sophie Clark	CONFIRMED
#7	2026-03-28	10:00	Michael Taylor	Grey Squirrel	Tom Baker	PENDING
#8	2026-03-28	13:00	Greenfield School	Feral Pigeon	Sophie Clark	CONFIRMED
#12	2026-03-29	09:00	Premier Restaurant Ltd	Pharaoh Ant	Rachel Green	PENDING
#13	2026-04-23	17:05	Abdul Adare	Carpet Beetle	Tom Baker	CONFIRMED

Recent Inspection Reports

REPORT	DATE	FINDINGS	FOLLOW-UP
#1	2026-03-20	Mouse droppings found behind kitchen units and under sink. E...	YES
#2	2026-03-21	Active rat burrow found near loading bay drainage. Gnaw mark...	YES
#3	2026-03-18	Follow-up visit. No new mouse activity detected. Bait statio...	NO

Data Structures and Algorithms

Binary Search Tree

Features

- Generic structure
- Efficient insert, search, delete
- In-order traversal

Complexity

- Insert/Search/Delete: O(log n) average
- Traversal: O(n)
- Space: O(n)

Testing Strategy

Framework

- MSTest
- TestClass and TestMethod usage

Coverage

- Repository testing
- Data structure testing
- AI agent testing
- Controller testing

Test Data

- Static repositories
- Deterministic data

Security Considerations

- SQL injection prevention
- XSS protection
- Input validation
- Proper HTTP responses

Performance Optimizations

Database

- Connection pooling
- Efficient queries

Frontend

- Parallel API calls
- Optimized DOM updates

Memory

- Singleton usage
- Proper disposal patterns

Conclusion

This project demonstrates:

- Clean architecture
- Strong use of design patterns
- Secure database interactions
- AI integration with Claude API
- Efficient frontend design

It represents a production-ready full-stack system combining traditional engineering with modern AI capabilities.

Appendix

Sample Data

- 10 customers
- 12 pest types
- 4 technicians
- Sample bookings and reports

Technology Stack

- Backend: ASP.NET Core 10.0, C# 12
- Database: SQL Server
- Frontend: HTML, CSS, JavaScript

- AI: Claude API
- Testing: MSTest

Development Environment

- Visual Studio / VS Code
- .NET SDK
- SQL Server Management Studio
- Web browser